SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO5 – Articulation	**Assessment:**	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	VIMALA K	**Reg. No.:**	192424276
Section / Batch:	B.Tech (AI&DS)	**Date:**	26/08/26

Code of Conduct

I ***VIMALA K 192424276*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: 

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

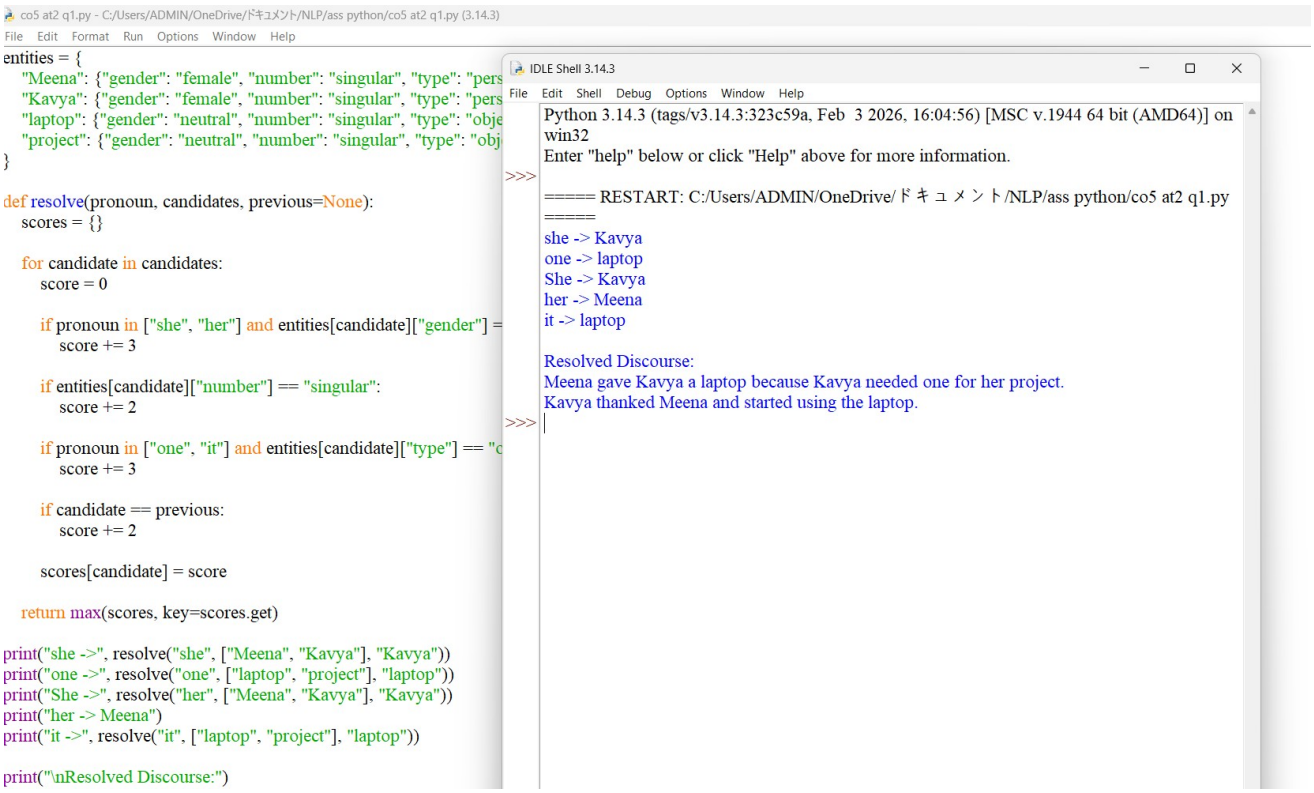
1. Identify all entities and referring expressions.
2. Generate possible antecedents for each pronoun.
3. Apply gender, number, recency, and semantic constraints.
4. Select the most appropriate antecedent.
5. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

Coding:

```
entities = {
    "Meena": {"gender": "female", "number": "singular", "type": "person"},
    "Kavya": {"gender": "female", "number": "singular", "type": "person"},
    "laptop": {"gender": "neutral", "number": "singular", "type": "object"},
    "project": {"gender": "neutral", "number": "singular", "type": "object"}
}

def resolve(pronoun, candidates, previous=None):
    scores = {}

    for candidate in candidates:
        score = 0

        if pronoun in ["she", "her"] and entities[candidate]["gender"] == "female":
            score += 3

        if entities[candidate]["number"] == "singular":
            score += 2

        if pronoun in ["one", "it"] and entities[candidate]["type"] == "object":
            score += 3

        if candidate == previous:
            score += 2

        scores[candidate] = score
```

```
    return max(scores, key=scores.get)

print("she ->", resolve("she", ["Meena", "Kavya"], "Kavya"))
print("one ->", resolve("one", ["laptop", "project"], "laptop"))
print("She ->", resolve("her", ["Meena", "Kavya"], "Kavya"))
print("her -> Meena")
print("it ->", resolve("it", ["laptop", "project"], "laptop"))

print("\nResolved Discourse:")
print("Meena gave Kavya a laptop because Kavya needed one for her project.")
print("Kavya thanked Meena and started using the laptop.")
```



2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

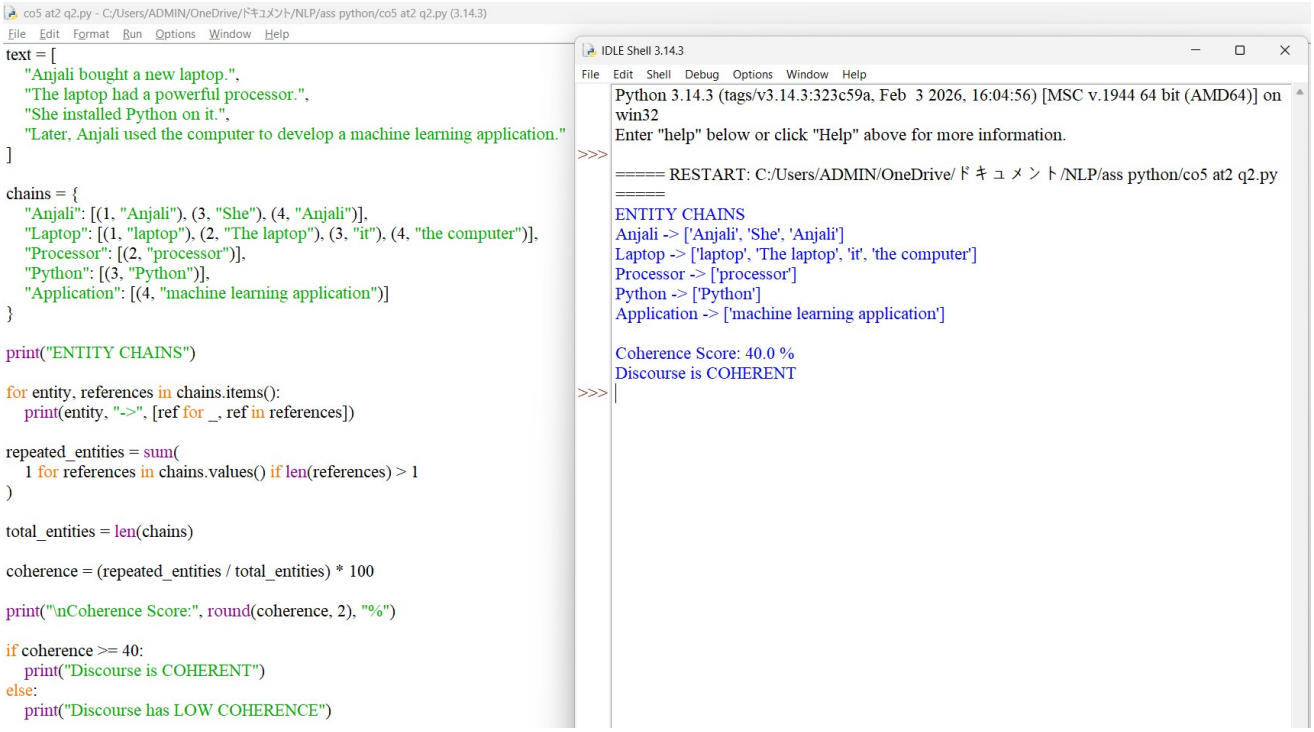
Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

Coding:

```
text = [
    "Anjali bought a new laptop.",
    "The laptop had a powerful processor.",
    "She installed Python on it.",
    "Later, Anjali used the computer to develop a machine learning application."
]

chains = {
    "Anjali": [(1, "Anjali"), (3, "She"), (4, "Anjali")],
    "Laptop": [(1, "laptop"), (2, "The laptop"), (3, "it"), (4, "the computer")],
    "Processor": [(2, "processor")],
    "Python": [(3, "Python")],
    "Application": [(4, "machine learning application")]
}

print("ENTITY CHAINS")



for entity, references in chains.items():
    print(entity, "->", [ref for _, ref in references])

repeated_entities = sum(
    1 for references in chains.values() if len(references) > 1
)

total_entities = len(chains)

coherence = (repeated_entities / total_entities) * 100

print("\nCoherence Score:", round(coherence, 2), "%")

if coherence >= 40:
    print("Discourse is COHERENT")
else:
    print("Discourse has LOW COHERENCE")
```



3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

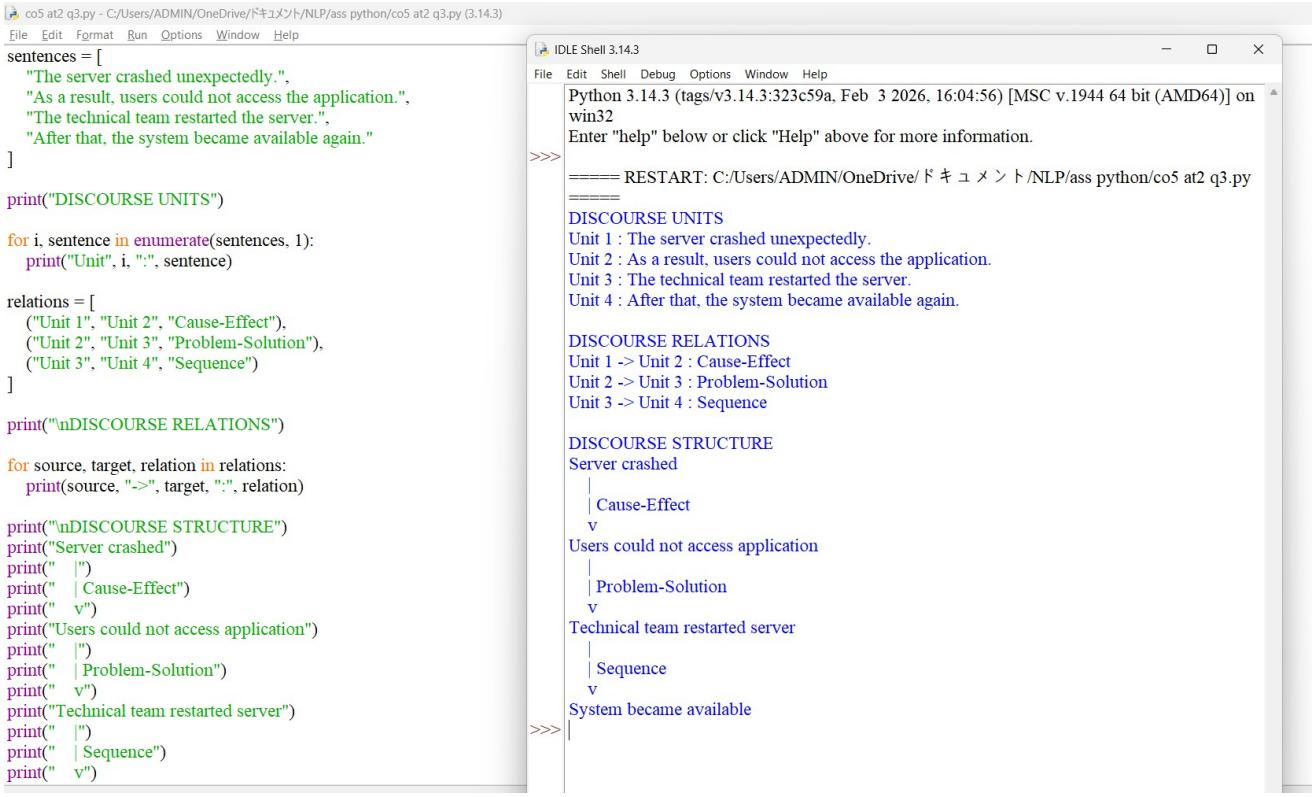
1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.
4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

Coding:

```
sentences = [
   "The server crashed unexpectedly.",
```

```
    "As a result, users could not access the application.",
    "The technical team restarted the server.",
    "After that, the system became available again."
]

print("DISCOURSE UNITS")

for i, sentence in enumerate(sentences, 1):
    print("Unit", i, ":", sentence)

relations = [
    ("Unit 1", "Unit 2", "Cause-Effect"),
    ("Unit 2", "Unit 3", "Problem-Solution"),
    ("Unit 3", "Unit 4", "Sequence")
]

print("\nDISCOURSE RELATIONS")

for source, target, relation in relations:
    print(source, "->", target, ":", relation)

print("\nDISCOURSE STRUCTURE")
print("Server crashed")
print("    |")
print("    | Cause-Effect")
print("    v")
print("Users could not access application")
print("    |")
print("    | Problem-Solution")
print("    v")
print("Technical team restarted server")
print("    |")
print("    | Sequence")
print("    v")
print("System became available")
```



4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.

Apply your algorithm to:

User: "Hello, I need help with my assignment."

Agent: "Sure, what problem are you facing?"

User: "I do not understand machine learning."

Agent: "I can explain the basic concepts to you."

Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

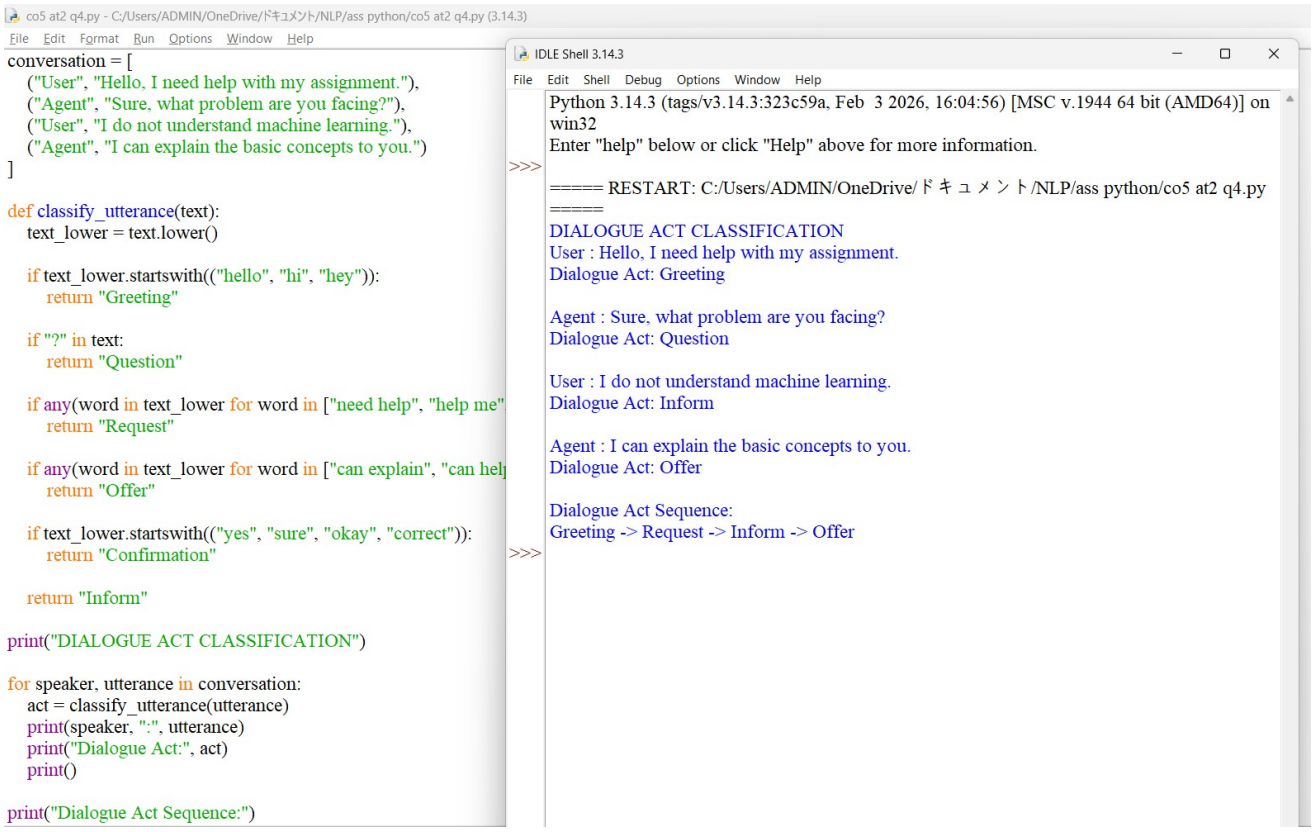
Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.
5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

Coding:

```
conversation = [
    ("User", "Hello, I need help with my assignment."),
    ("Agent", "Sure, what problem are you facing?"),
    ("User", "I do not understand machine learning."),
    ("Agent", "I can explain the basic concepts to you.")
]

def classify_utterance(text):
    text_lower = text.lower()

    if text_lower.startswith(("hello", "hi", "hey")):
        return "Greeting"

    if "?" in text:
        return "Question"

    if any(word in text_lower for word in ["need help", "help me", "please"]):
        return "Request"

    if any(word in text_lower for word in ["can explain", "can help", "i can"]):
        return "Offer"

    if text_lower.startswith(("yes", "sure", "okay", "correct")):
        return "Confirmation"



    return "Inform"

print("DIALOGUE ACT CLASSIFICATION")

for speaker, utterance in conversation:
    act = classify_utterance(utterance)
    print(speaker, ":", utterance)
    print("Dialogue Act:", act)
    print()

print("Dialogue Act Sequence:")
print("Greeting -> Request -> Inform -> Offer")
```



5. Algorithm for Dialogue State Tracking

Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.

Apply your algorithm to:

User: "I want to book a flight."
Agent: "Where would you like to travel?"
User: "I want to go to Delhi."
Agent: "From which city?"
User: "From Chennai."

Your algorithm should maintain the following slots:

Departure City
Destination City
Travel Date
Number of Passengers

Perform the following steps:

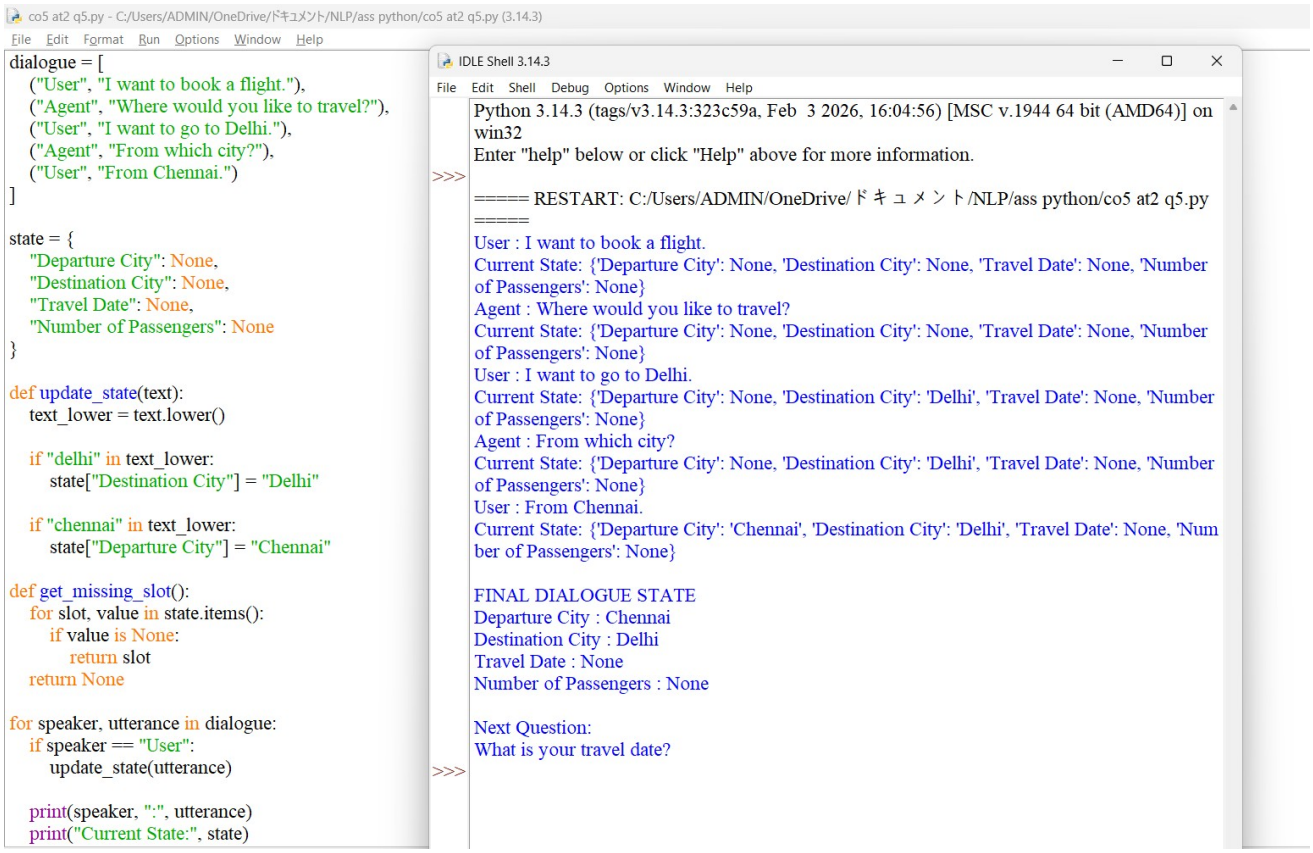
1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.
5. Generate the next appropriate question.

Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

Coding:

```
dialogue = [
    ("User", "I want to book a flight."),
    ("Agent", "Where would you like to travel?"),
    ("User", "I want to go to Delhi."),


    ("Agent", "From which city?"),
    ("User", "From Chennai.")
]

state = {
    "Departure City": None,
    "Destination City": None,
    "Travel Date": None,
    "Number of Passengers": None
}

def update_state(text):
    text_lower = text.lower()

    if "delhi" in text_lower:
        state["Destination City"] = "Delhi"

    if "chennai" in text_lower:
        state["Departure City"] = "Chennai"

def get_missing_slot():
    for slot, value in state.items():
        if value is None:
            return slot
    return None

for speaker, utterance in dialogue:
    if speaker == "User":
        update_state(utterance)

    print(speaker, ":", utterance)
    print("Current State:", state)

print("\nFINAL DIALOGUE STATE")

for slot, value in state.items():
    print(slot, ":", value)

missing = get_missing_slot()
```

```
print("\nNext Question:")

if missing == "Travel Date":
    print("What is your travel date?")


elif missing == "Number of Passengers":
    print("How many passengers are travelling?")
elif missing:
    print("Please provide your", missing)
else:
    print("All required information has been collected.")
```



Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1	4	4	4	4	3	19	88
2	4	3	4	3	3	17	
3	4	3	4	4	3	18	
4	4	3	4	3	3	17	
5	4	3	4	3	3	17	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____